Point-in-Time Correctness

as a Test of Data-Science Agents

Why identical failures across samples do not prove a capability gap

Adit Rahman

Research Project

June 2026, revised October 2026

Abstract. Five tasks ask an agent to rebuild what a data warehouse knew at a past moment. Each instruction states the goal and the principle, use only what the business knew at that instant, and leaves the agent to work out the rule that makes it true: a fact counts only if it both happened and was recorded before the decision. `gemini-3.5-flash`, run twelve times on each task, scores pass@1 21.7% and pass@3 25.0%, and on every failing task it fails on the same cells in every run. The trajectories tell a different story from the scores. The model applied the unstated rule in all sixty runs. Two gaps are real: it does not extend the rule to records its own system produced, and it does not look for duplicate rows unless told. The other zeros are conventions my instructions left open, which the model read the other way, identically, twelve times out of twelve. A model that reads deterministically fails an open convention exactly as consistently as it fails a skill it lacks, so identical failures across runs, the signature one designs for, cannot by themselves separate the two.

1 Introduction

A data scientist who builds a training table does one thing under every row: reconstruct what was knowable at that row's moment. A credit decision made in March may use a February transaction only if the warehouse had recorded it by March. A churn label may be assigned only if its outcome window has closed within the data. Get this wrong and the model scores well offline and fails in production. Kaufman, Rosset and Perlich rank leakage among the top data-mining mistakes; Kapoor and Narayanan found it in 294 published papers across 17 fields; feature stores such as Feast, Tecton and Databricks exist in large part to prevent it.

I wanted to know whether a frontier model, acting as an agent in a shell with pandas, has this skill, and whether it applies it when the instruction states the goal and the principle but not the method, on data with real warehouse mess. My expectation came from a picture of agents as template matchers: an agent maps the problem in front of it onto problems it has seen, and the seen problem becomes the plan. Without a template it falls back to the nearest thing it can do, usually the easier, wrong implementation. The point-in-time join is described everywhere in public text and solved mostly inside proprietary systems, so I expected vocabulary without reflex.

The measurement bar was pass@3 under 30%: pass@k is the chance that at least one of k independent attempts at a task passes, averaged over tasks. That bar constrains the design. If attempts fail independently with per-attempt success p, pass@3 is $1-(1-p)^3$, and $p = 0.11$ already clears 30%. A low pass@3 means something only when the attempts fail the same way, so each task was built around one withheld mechanism, a rule the data requires and the instruction does not state, with a plausible wrong reading next to the correct one. The model did take the wrong reading every time. What the wrong reading was is the subject of this report.

Related work shaped the design more than the conclusions. DataClawBench (492 tasks over 2.06M records, native noise kept, no schema hints; best of eight agents under 50%) is where the messy-inputs, no-method-hints, per-milestone scoring approach comes from. DSAEval, DataSciBench, InfiAgent-DABench, DSBench, DA-Code and MLE-bench frame the wider landscape. SWE-bench Verified supplied the hidden-gold verifier. Flash is not on the DataClawBench leaderboard; Gemini 3.1 Pro is, and Flash beats it on Terminal-Bench 2.1 under the same Terminus-2 harness used here (76.2% to 70.3%).

2 Method

Every task is point-in-time reconstruction over bitemporal data. Bitemporal means each fact carries two timestamps: its event time, when it happened or took effect, and its recorded time, when the warehouse received the row. A fact is knowable at a moment if both timestamps are before that moment. A feature or label computed as of a moment may use only knowable facts (Figure 1). Filtering on event time alone admits facts that happened early and were recorded late. An as-of join is the operation that, for each output row, picks the latest knowable record as of that row's own timestamp; it is the standard tool and, used naively on one timeline, the standard mistake.

Each of the five tasks is a scaled-down version of a job a data scientist does, in a different industry (Table 1). The agent receives an instruction and a directory of input files with a data dictionary that documents every table and column. The instruction says what the output table is for, states the point-in-time principle in plain words, and defines every output column. It does not name the recorded-time filter, the as-of rule, the duplicates or the casing. Those are the withheld mechanisms, and the data is built so that the naive reading and the correct one disagree on many rows: a trap, in the vocabulary below, is a planted case of that disagreement, such as a transaction that happened before a decision but was recorded after it. The inputs also carry ordinary warehouse mess: double-ingested rows, mixed casing, three timestamp encodings. The largest input has about

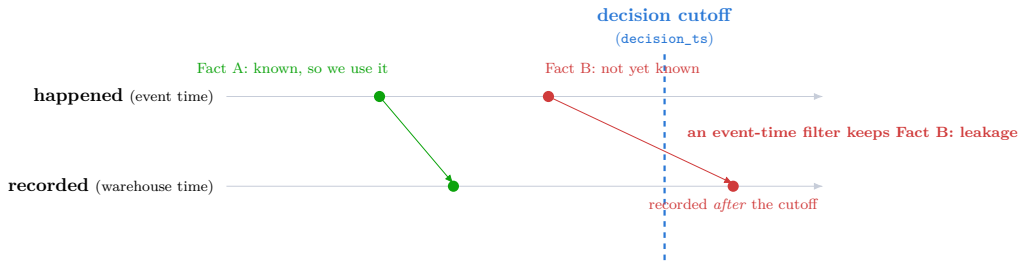


Figure 1: Fact B happened before the decision but was recorded afterwards, so it was not knowable at decision time. A correct as-of join filters on both timelines.

6,300 rows. Two more words recur. The headroom of a task is the room between how the model performs on it and what is achievable, the thing an eval is meant to measure. The anchor is a task included because the model was expected to solve it, to mark the bottom of the difficulty range.

Task	**Domain**	**What the agent builds**	**Withheld mechanism**
credit	Consumer credit	Feature table for a credit-risk model: one row per past lending decision, eight features as known at that moment, from transactions, account snapshots and disputes.	Recorded-time filter on every source; latest knowable snapshot; drop duplicate rows.
claims	Health insurance	Member cost share per past claim as known at adjudication, including a year-to-date deductible accumulator over the member's earlier claims.	Enrollment as recorded before adjudication; knowability applied to the accumulator.
churn	B2B SaaS	Churn labels: did the account churn within 90 days? Observations whose window has not closed get no label.	Right-censoring; labels by effective time, not recorded time.
ledger	E-commerce	Each order's state at past checkpoints, folded from an append-only event log where later corrections void earlier events.	Knowability on both timelines; a correction voids its target only once it lands.
twr	Finance reporting	Cumulative time-weighted return per account at each month-end, from clean daily valuations and cash flows. The easy anchor.	Strip the day's flow before chaining growth factors.

Table 1: The five tasks. Each took an expert an estimated 1.5 to 2.5 hours.

A task earned a place only if one global filter plus a standard as-of join cannot solve it: it needs a per-row cutoff, two timelines that must both hold, and a sum or count that is wrong if duplicates survive. Eleven candidates were piloted; five kept. Four of the six cut were solved 3 of 3 at first pilot, because the data dictionary or the instruction stated the method, or because a single global snapshot reduced the task to one filter and one join. One was a verifier bug: three trials missed six of 19,565 cells by exactly one cent against a tolerance sitting on the float boundary. The sixth, a smart-meter billing task, scored 0 of 12 and looked like headroom until I read the trajectories. In twelve of fifteen trials the only failing cells were the 112 bills with a meter read stamped exactly at the bill instant, which the model read as known and the verifier as not yet known. Pinning that convention, and nothing else, took it to 3 of 3. The lesson sat unapplied to the kept set for a while.

Each task runs in Harbor, a framework that gives the agent a Docker container holding the inputs, passes it the instruction, and after the agent finishes runs a verifier in a separate container. The verifier recomputes the correct output table, the truth, from a pristine copy of the inputs, and compares the agent's table to it cell by cell, a cell being one value at one row and column. It runs structural checks first (exact column names and order, one row per key, sorted), then one check per computed column with a tolerance that forgives rounding (0.01 on money, 0.5 on counts, 10^{-3} on ratios) and not a point-in-time mistake, which moves values by whole counts and whole dollars. The reward is 1 if every check passes, else 0. Binary, because the structural checks are easy and a fractional score would mostly measure how much scaffolding I counted; a program rather than an LLM judge, because the answer is exactly computable. Each task carries at least one control column, a column whose value does not depend on the withheld mechanism, so that a naive method gets it right and a failure there would indicate a broken task rather than a missed rule. A reference solution scores 1 and a do-nothing agent scores 0 on every task, and the generator, truth and tests never enter the agent's container.

The agent is Terminus-2 driving `gemini-3.5-flash`. Terminus-2 is a terminal scaffold: at each turn the model returns a JSON object holding its analysis and a batch of shell commands, the scaffold runs the commands and returns the terminal output, and the loop continues until the model declares the task complete. The container offers a shell and Python with pandas and pyarrow, no network. A trial is one complete run of the agent on one task; a step is one turn within it, and the trajectory is the recorded sequence of steps. When the model's turn is not valid JSON the scaffold rejects it and asks again, so turn counts include these retries. Twelve trials per task, sixty in all, $41 of API spend. pass@$k$ is the unbiased estimator $1 - \binom{n-c}{k}/\binom{n}{k}$ with $n = 12$ trials and c passes.

After the scores were in, I audited every trial: extracted the final script that wrote the output, recomputed the truth under each plausible reading of each open point in the instruction, and found the reading that reproduces the trial's output on every cell. Every trial's output is explained exactly by one rule set. Then I read the trajectory to the step where the model chose that rule. Quotes are verbatim and cited as (trial id, step), indexing the released run logs. Claude Code did the extraction, recomputation and matching; which readings a careful expert could take from the instruction is my call.

3 Results

Task	**Passed**	**pass@1**	**pass@3**	**Turns**	**Wall**
credit	0/12	0%	0%	27	180 s
claims	0/12	0%	0%	40	278 s
churn	0/12	0%	0%	31	183 s
ledger	1/12	8.3%	25.0%	40	269 s
twr	12/12	100%	100%	18	90 s
Mean over tasks	13/60	**21.7%**	**25.0%**		

Table 2: Twelve trials per task, no crashes or timeouts. Turns and wall time are medians per trial; turns include harness retries.

Four tasks sit at or near zero and the anchor is solved every time (Table 2). The mean pass@3 is 25.0%; it crosses 30% only at $k = 6$, and only through the ledger's single pass (Figure 2). The three zero tasks are flat at every k. The model spent more turns and more time on the tasks it failed than on the one it solved, and every trial produced a well-formed table.

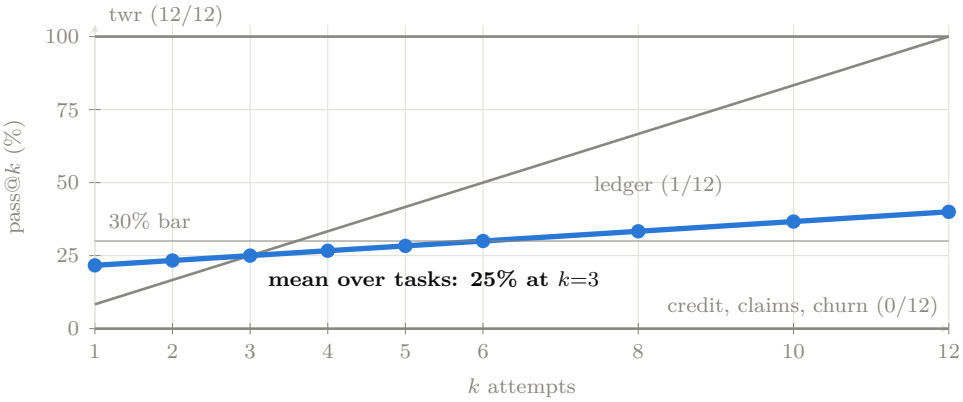


Figure 2: pass@k for $k = 1 \ldots 12$. The blue line is the mean over the five tasks.

Figure 3 shows every output column and how many of the twelve trials got it wrong. The control columns pass everywhere. One mechanism per task fails in every trial. Within a task the wrong cells are the same cells: all 12 churn trials miss the identical 14 rows; 11 credit trials miss the identical 169 transaction rows and all 12 the identical 104 snapshot rows; seven ledger trials miss the identical 121 cancellation rows and eight the identical 62 supersession rows. The model makes one decision per question and makes it the same way every time, which is exactly the behaviour the tasks were designed to elicit.

4 What the trajectories show

The audit sorts every wrong cell into one of two kinds. A gap is a rule the instruction and data support that the model did not apply. A fork is a point where the instruction admits two readings a careful expert could each defend, the verifier encodes one, and the model chose the other. The distinction is about the instruction, not the model: the same deterministic misreading produces the same cells either way.

4.1 Churn

The task gives a log of subscription events (signup, renewal, cancellation, reactivation), each with the time it took effect and the time it was recorded, and a set of observations, each an account and a timestamp. For each observation the agent reports whether the account was a live subscriber at that moment, whether a 90-day churn label can be assigned, and the label. The log is complete only up to a stated cutoff date, so an observation whose 90-day window runs past the cutoff has no observable outcome and gets no label; that is right-censoring. A cancellation followed by a reactivation inside the window is a win-back, not a churn. Labels describe what actually happened, so they are read by effective time: a cancellation recorded late still counts from the day it took effect.

Every trial handled right-censoring (278 immature rows), win-back, the inclusive boundary, duplicates and fifteen casing variants. None filtered events by recorded time, which was the trap I expected to catch the most trials, and several argued the point: *"Under the complete history, the state is indeed canceled from Oct 31, 2024 onwards"* (`E9vLsjs`, step 28). The 14 failing rows are accounts already cancelled before the observation, with a matured window. The truth says ineligible, blank; the model says eligible, label 0.

All twelve trials noticed the sentence *"An account that was not a live subscriber at the observation cannot churn from it"*, debated it, and landed in the same place: *"This is about the label being 0*

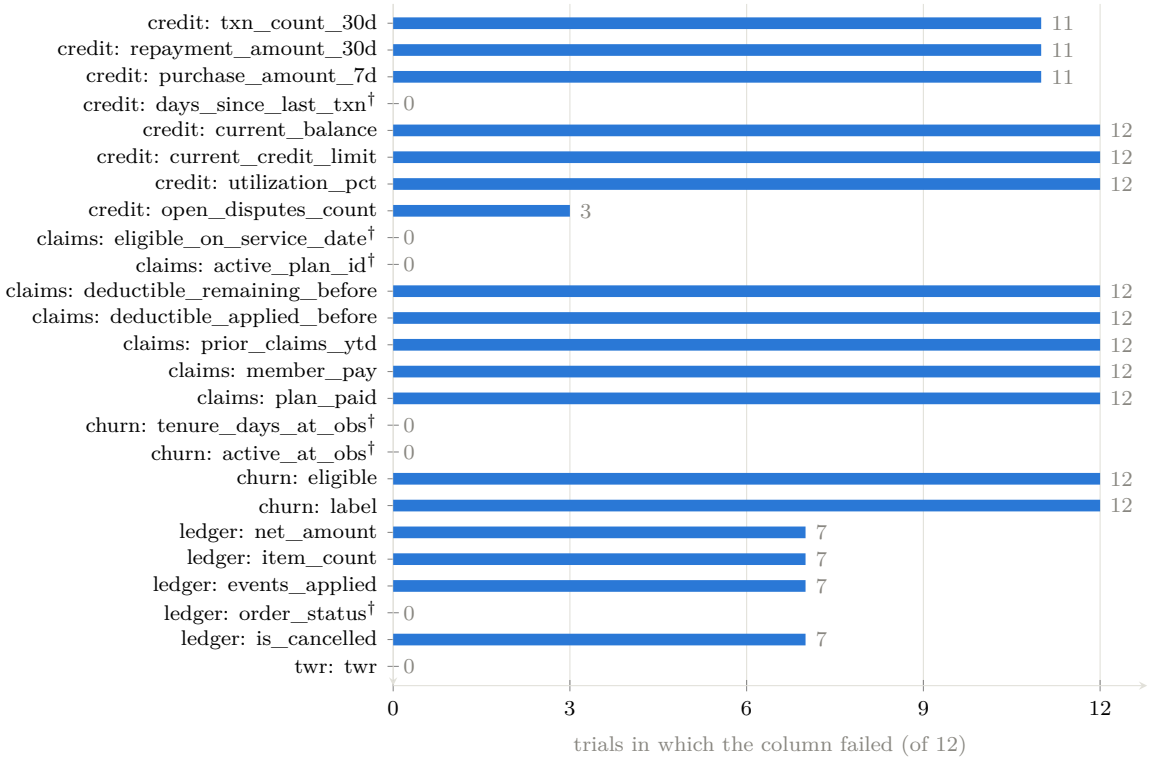


Figure 3: Per-column failure counts over twelve trials, from the verifier's per-test output. † marks a control column that a naive method also gets right.

because they can't churn if they aren't active. It doesn't mean they are ineligible. The eligibility condition is defined separately" (`DexxzHE`, step 9). Five named the survival-analysis convention I had in mind and rejected it because the eligibility clause names only the time condition. Five printed the 14 rows and approved them. The instruction never ties eligibility to being active. The reading is literal and defensible, and the whole 0 of 12 rests on it.

4.2 Ledger

A marketplace stores each order as an append-only stream of events (placed, item added or removed, payment captured, fulfilled, refunded, cancelled), each carrying a change to the order's value and item count, an event time and a recorded time. A correction event restates an earlier event: the dictionary says it names the event it supersedes, and once the correction is knowable the original no longer counts. A checkpoint is an order and a timestamp; for each one the agent reports the order's net value, item count, number of contributing events, lifecycle status (set by the latest status-changing event), and a flag `is_cancelled`.

All twelve trials implemented strict knowability on both timelines, discovered supersession from the word *"restates"* within the first few steps, kept the original event standing until its correction became knowable (*"we MUST count the superseded event, because the correction hasn't happened or hasn't been recorded yet!"*, (`p6UZuxj`, step 5)), deduplicated, and ordered status by event time. The status column passed 12 of 12.

The 121 failing cancellation cells are orders cancelled and then fulfilled or refunded. The truth treats `is_cancelled` as sticky: once an order has a knowable cancellation, the flag stays set whatever happens next. Seven trials read *"has been cancelled as of the checkpoint"* as current status, which is what the same phrase means for the status column two rows above it. Three stated the

sticky reading first and switched: *"The existence of a separate boolean column strongly implies that `is_cancelled` might be different"* (`ry5KWCc`, step 25), then *"the most standard interpretation of current cancellation state"* (`ry5KWCc`, step 44). One tabulated the exact 121 disputed rows before choosing the other side (`JQRqxaY`, step 33).

The 62 supersession cells are events with two knowable corrections. The truth counts both; eight trials kept the latest. *"If we replaced A with both B and C, we would have two events replacing one, which might mean we double-count the corrected item!"* (`VATRPZT`, step 17). Nothing in the instruction or dictionary addresses the case; the generator produced it as a side effect. The one passing trial landed on the truth side of both forks, the second one by not noticing the case existed. The most careful trial in the set, which hand-traced four checkpoints and argued the sticky reading correctly, failed because it investigated the multiple-correction case and resolved it.

4.3 Credit

For each past lending decision, identified by a customer and a timestamp, the agent computes eight features from three sources: transactions (purchases and repayments, with a 30-day count and sums over 30 and 7 days), periodic account snapshots that carry balance and credit limit and stay in force until the next snapshot, and disputes that are open from when they are raised until they are resolved. Each source row has an event time and a recorded time. The data plants boundary rows, records whose event time equals a decision's timestamp exactly, in all three sources.

All twelve trials filtered transactions, snapshots and disputes on recorded time and on event time, and all twelve excluded the 190 transactions stamped exactly at the decision. The late-recorded snapshot, the trap the task was built around, was reasoned out in so many words: *"is it possible that `snapshot_ts <= decision_ts` but `recorded_ts > decision_ts`? Yes! If that's the case, we can't use that snapshot; we must look at the previous latest snapshot where both conditions hold"* (`EozkuP8`, step 9). No wrong cell in twelve trials comes from the knowability rule.

The 169 wrong transaction counts in eleven trials are exactly the 169 decisions with a duplicate row inside the window; the error distribution (+1: 118, +2: 45, +3: 5, +4: 1) is the duplicate-row distribution. Ten trials never checked. One checked and kept them: *"there are exactly 356 duplicates of entire rows! … No, the instructions do not say we should de-duplicate"* (`yxZsqmU`, step 15). One checked and dropped them: *"This strongly indicates that these duplicates are data ingestion artifacts and should definitely be deduplicated!"* (`4wHj44Y`, step 32). The dictionary says the transaction id is unique.

The 104 wrong snapshot cells in all twelve trials are exactly the 104 decisions with a snapshot stamped at the decision instant. The instruction says the balance *"in effect as of the decision"* and states an exclusive boundary only for the transaction window. The model counted the boundary rows and chose: *"Wow, 104 snapshots have `snapshot_ts == decision_ts`! … since it represents the state at the decision, it should definitely be included"* (`4wHj44Y`, step 52). The verifier wanted strictly before; SQL `AS OF` semantics are inclusive. The same trial measured both dispute conventions (704 versus 634 open disputes), switched from the verifier's reading to the other, and lost 70 cells for it. With both boundaries accepted, one trial passes and eleven fail on duplicates alone.

4.4 Claims

A health insurer adjudicates each claim, that is, prices it, at a known timestamp after the date of service. For each claim the agent determines whether the member was covered on the service date, from enrollment spans that can be added or corrected after the fact (a termination backdated to before the service date, recorded after the claim was adjudicated, must not count), and then computes the member's share of the claim's allowed amount under a deductible-then-coinsurance rule. The deductible part depends on an accumulator: the total allowed amount of the member's

earlier claims in the same calendar year that were already known at this claim's adjudication. A claim for a member who was not covered is denied and the member pays nothing.

The backdated termination was handled by all twelve trials: eligibility and plan id are 2,390 of 2,390 correct in every trial. The accumulator is wrong in every trial, on 197 to 1,420 rows; the money columns show it on fewer rows only because most members had already met the deductible.

Nine trials did not apply the recorded-time filter to prior claims, which the instruction defines as claims *"already known at this claim's adjudication"*. All nine built the same argument. *"The payer itself is the one who adjudicated … So Option A (adjudicated earlier) is the only option that makes logical business sense for how insurance claims are adjudicated in real-time"* (`8ZE3vUz`, step 28). That trial had just measured the two options, 1,881 against 1,740 claims with a prior total, and chose the leaky one. Another counted *"pairs where prev adj < curr adj but prev rec > curr adj: 1992"* (`J4PeZiG`, step 24) and chose it too. The model applied bitemporality to the enrollment feed, which arrives from outside, and treated the claims table as the payer's own system of record, exempt from the warehouse clock. The instruction frames knowledge as what has *"reached the payer"*, which gives the argument a foothold; the dictionary says every table records when the row landed in the warehouse. Of everything in the set, this is the gap I trust most.

Two forks sit underneath it. All twelve trials excluded denied prior claims from the accumulator (*"A denied claim never counts towards meeting the deductible"*, (`Ew7ZYGT`, step 31)); the reference includes them, and the instruction does not say. Nine ordered *"earlier"* by adjudication rather than service date. The one trial that applied the knowability filter, used service order and deduplicated fails only on the denied-claims fork plus seven cells one cent off at the tolerance boundary.

4.5 TWR

The anchor asks for each brokerage account's time-weighted return at each month-end: the return the investments earned, neutral to the client's deposits and withdrawals. The inputs are clean daily closing values V_t and external cash flows C_t that settle at the close and are already inside V_t. The correct method chains daily growth factors $(V_t - C_t)/V_{t-1}$; the instruction names the measure and the flow timing and says nothing about the method.

In eight of twelve trials that growth factor appears in closed form in the first reasoning turn after inspecting the files, before any computation, with the textbook vocabulary attached: *"This is also called the 'Daily Valuation Method' or the 'true Time-Weighted Return"'* (`ec7hk6y`, step 12); *"our calculation is 100% GIPS-compliant"* (`GUoHvZC`, step 14), GIPS being the investment industry's performance-reporting standard. The only alternative that ever appeared was the start-of-day timing variant, raised in order to reject it on the dictionary's word *"close"*. No trial considered a simple or money-weighted return. All twelve outputs are numerically identical.

4.6 Summary

Table 3 collects the mechanisms. Every one I withheld on purpose was implemented in twelve of twelve trials. Of the nine the model got wrong somewhere, two are gaps and seven are conventions the instruction left open.

5 Discussion

The design goal was a set where the model fails the same way every attempt, and it was met on every failing task. But a model that reads every open convention deterministically produces that signature whether the cause is a missing skill or a sentence the author left open. The reward did not distinguish the two; the per-column counts did not; the cell clustering did not. Neither did the checks one usually runs instead: a reference solution that must score 1, a do-nothing agent that must score 0, and a rubric judge, an LLM that reads the task files against a task-quality checklist,

Task	Mechanism	Right (of 12)	Verdict
credit	recorded-time filter on all three sources	12	cleared
	fallback to earlier snapshot when the latest landed late	12	cleared
	strict event-time window boundary; casing	12	cleared
	drop duplicate transaction rows	1	**gap**
	snapshot at exactly the decision instant: in effect or not	0	fork (104 rows)
	dispute resolved at exactly the decision instant: open or not	9	fork (70 rows)
claims	retroactive enrollment: use spans recorded before adjudication	12	cleared
	dedup; calendar-year reset; deductible waterfall	12	cleared
	recorded-time filter on the member's *own* prior claims	3	**gap**
	"earlier" = by service date, not adjudication order	3	soft fork (133 rows)
	denied prior claims still count toward the deductible	0	fork (197 rows)
churn	right-censoring at the data cutoff	12	cleared
	labels by effective time, no recorded-time censoring	12	cleared
	win-back; inclusive 90-day boundary; dedup; casing	12	cleared
	not a live subscriber at observation ⇒ ineligible	0	fork (14 rows)
ledger	knowability on both timelines, strict	12	cleared
	correction supersedes its target; late correction invisible until it lands	12	cleared
	dedup; casing; status by event time	12	cleared
	`is_cancelled` is sticky, not current status	5	fork (121 rows)
	two corrections of one event both count	4	fork (62 rows)
twr	flow-neutral daily chaining; data-driven month-ends	12	cleared

Table 3: Per mechanism, the number of trials whose final code implemented the rule the verifier wanted. A **gap** is a rule the instruction and data support that the model did not apply. A fork is a convention the instruction left open, where the model took the other defensible reading. Row counts are the cells the choice flips, per trial.

all passed every task here, forks included. What distinguished them was recomputing the truth under the model's reading and seeing whether its output matched exactly. It always did.

The model's tie-breaks are consistent and often opposite to the author's. On boundaries it reads *"as of"* as inclusive. On flags it reads *"has been X as of"* as current state. On accumulators it excludes rows a domain expert would exclude. On its own records it trusts the operational system over the warehouse. None of this moved with temperature across twelve samples. An author who wants the other convention has to write it down, and an author who does not know which conventions are open will not learn it from the scores.

Nor from the model. Every trial spot-checked a row, almost always the first row, by re-running its own filter by hand. In credit, ten of twelve hand-checked the first decision, whose window contains the duplicate transaction; three had both copies on screen and one declared *"Manual: 16. Match!"* against a truth of 15 (`EozkuP8`, step 22). In claims, one trial wrote a semantic self-test, reported *"Total differences found: 0"*, and declared the ambiguity eliminated (`TFRWhSz`, step 26); the test was wrong, and the truth has 345 differences. In TWR, a trial quoted sixteen-digit intermediate

values that had never been printed, found a discrepancy between two of its own fabrications, and resolved it with a tool call (`44YNQQX`, step 15). Final confidence was uniform (*"flawless"*, *"100% correct"*, *"absolutely perfect"*) and uncorrelated with correctness.

So much for vocabulary without reflex. The reflex for the canonical pattern is there: filter each source on both clocks, pick the latest knowable state, let a correction void its target once it lands. That pattern exists in the training data in solved form; feature-store documentation and many tutorials contain it. What the model lacks is narrower. It does not extend the rule to records its own system emitted. It does not look for duplicates unless warned: it deduplicated in twelve of twelve trials on the three tasks whose dictionary mentions double ingestion, and in one of twelve on the task whose dictionary only says the id is unique. And it resolves every under-specified convention by its own prior and then reports certainty. Those are the missing templates, and a set built to find headroom in this skill should be built around them rather than around the as-of join itself.

Measured, the set clears the 30% pass@3 bar at 25%. With the open conventions pinned (strictly-before stated for every boundary, *"known"* defined as recorded in the warehouse for every table, eligibility tied to being active, cancellation stated as never undone, the two-corrections case removed from the generator or specified), churn and most ledger trials should pass, credit should stay near zero on duplicates alone, and claims should stay near 3 of 12 on knowability. That set would not clear the bar without new tasks, and the new tasks should target the two gaps: knowability of self-produced records in accumulators and running balances, and hygiene the instruction does not request.

At scale, volume is the cheap part; each task is a seeded generator plus a verifier that recomputes the truth, and the mechanism, the industry domain and the noise in the inputs are independent axes. The step that separated gaps from forks is the one to make routine: for each failing column, recompute the truth under each plausible alternative convention, and if one reproduces the model's output exactly, pin the convention or accept both readings before the task counts. That is a loop over candidate rules and a diff; the trajectory read that follows has the narrow job of locating the decision and classifying it.

6 Limitations

All sixty trials use one model under one scaffold, so I cannot say how the two gaps behave under a stronger model. The inputs are synthetic; the generators model real warehouse mess and are seeded, but a real warehouse has mess I did not model, and the audit found one generator bug (two churn events effective after the stated cutoff, harmless to the truth). Twelve trials bound per-attempt success under 22% at 95% confidence for the zero tasks. Calling a reading a fork is a judgement about what a careful expert could take from the instruction and dictionary; the ledger and churn forks are clear, and the claims knowability case is the closest call. Step counts are upper bounds on reasoning effort, since JSON retries inflate them.

7 Conclusion

A frontier model has the canonical point-in-time reflex and two specific blind spots: it exempts its own system's records from the warehouse clock, and it does not look for duplicates unprompted. It also resolves every open convention by its own prior, the same way every time, with full confidence, which is why most of the zeros in this set were mine. Designing an eval for correlated failure produces the pass@k signature one wants; the signature cannot tell a missing skill from an open sentence, and the step that can is cheap.